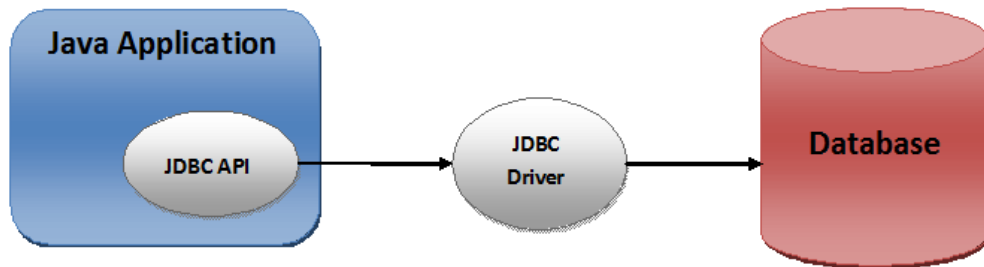


Java Database Connectivity (JDBC)

What is JDBC ?

- JDBC stands for Java Database Connectivity.
- It is an API that allows Java programs to connect and interact with databases.
- JDBC is used to execute SQL queries like insert, update, delete, and select.
- It acts as a bridge between the Java application and the database.



Purpose of JDBC :-

- JDBC is used to connect Java applications with databases.
- It helps execute SQL queries from Java programs.
- It allows inserting, updating, deleting, and retrieving data.
- It provides a standard way for database communication.

Prerequisites of JDBC :-

Before writing JDBC code, make sure you have:

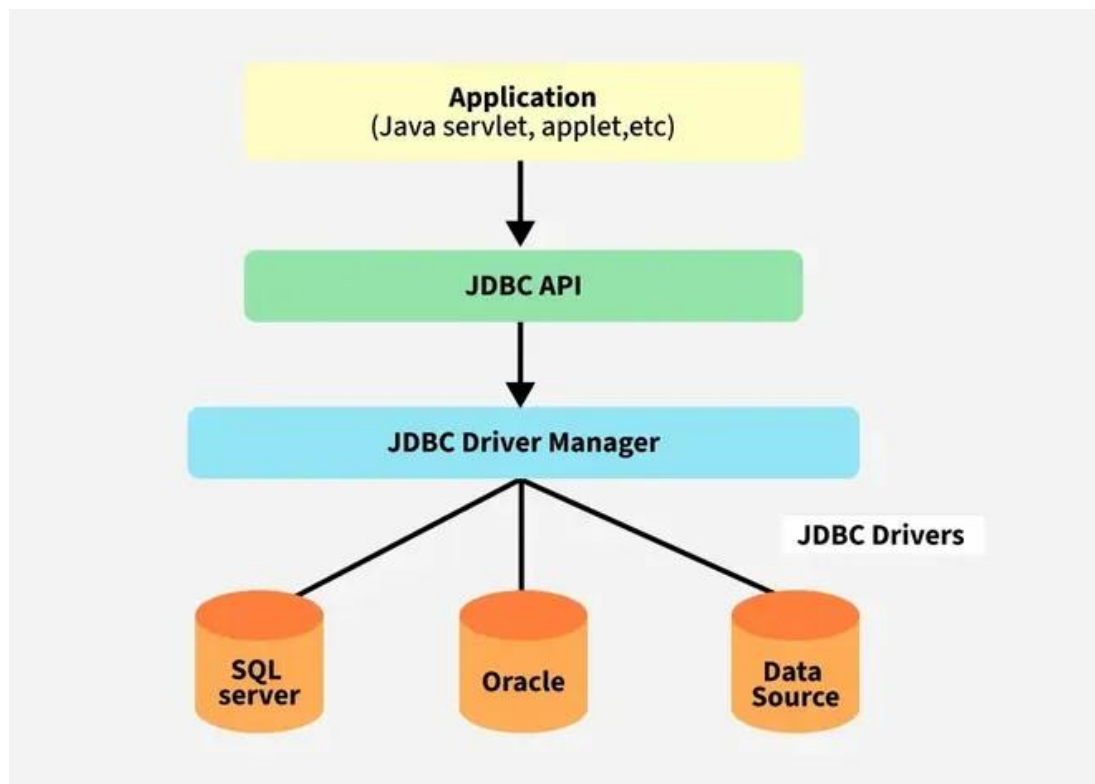
1. Database Installed and Running:

- Install a relational database (e.g., PostgreSQL)
- Create a database and a user.
- Set username, password & portnumber

2. JDBC Driver (Connector JAR):

- JDBC requires a driver to connect with the database.
- Download the JAR or add Maven dependency.

JDBC Architecture :-

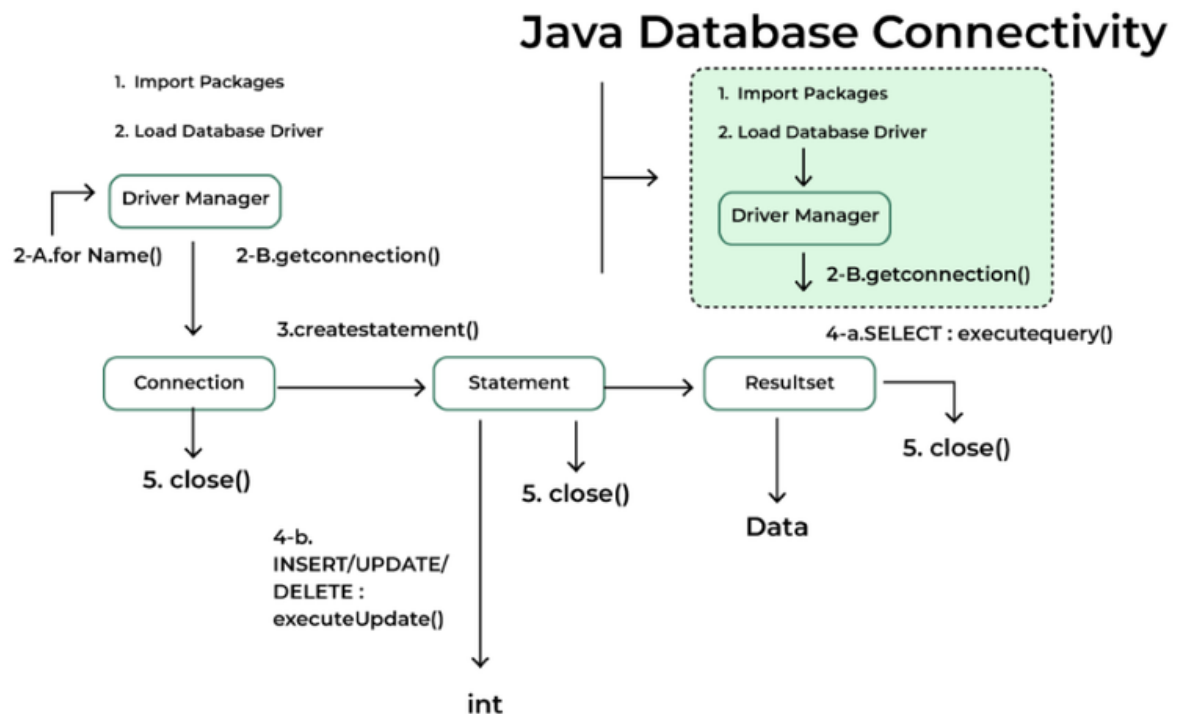


Components of JDBC -

- 1. Application:** It can be a Java application or servlet that communicates with a data source.
- 2. JDBC API:** It allows Java programs to execute SQL queries and get results from the database. Some key components of JDBC API include
 - Interfaces like Driver, ResultSet, RowSet, PreparedStatement and Connection that helps managing different database tasks.
 - Classes like DriverManager, Types, Blob and Clob that helps managing database connections.
- 3. DriverManager:** It plays an important role in the JDBC architecture. It uses some database-specific drivers to effectively connect enterprise applications to databases.
- 4. JDBC drivers:** These drivers handle interactions between the application and the database.

Steps to Establish a JDBC Connection :-

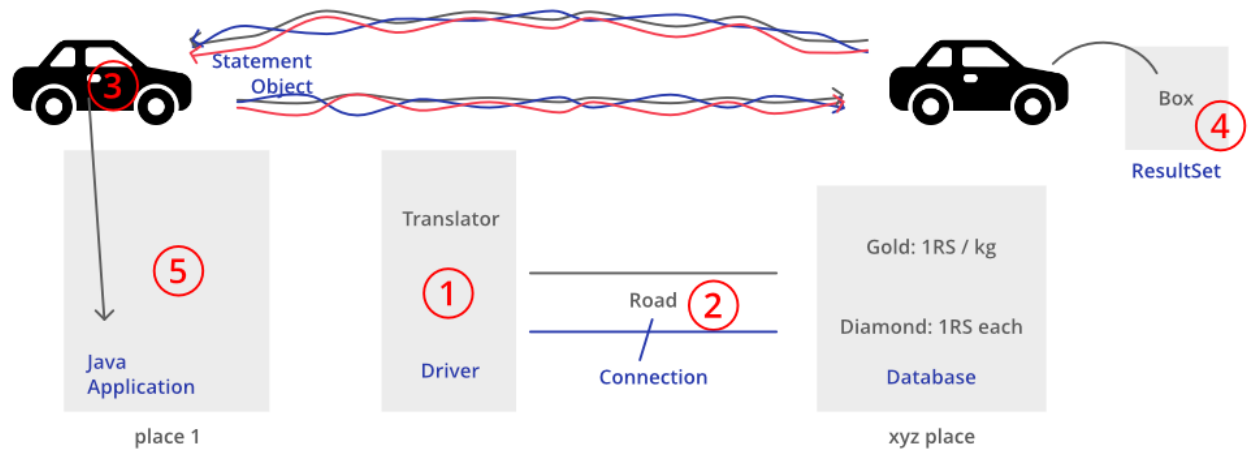
To perform database operations using JDBC, follow these standard steps:



1. **Import JDBC package:** Include java.sql classes for database operations.
2. **Load and register driver:** Load the JDBC driver class to enable communication using forname() method
3. **Establish connection:** Use DriverManager to connect to the database.
4. **Create statement:** Create Statement or PreparedStatement to send SQL queries.
5. **Execute query:** Run SQL commands like SELECT, INSERT, UPDATE or DELETE using
6. **Process results:** Retrieve and handle data from ResultSet.
7. **Close resources:** Release database connections and objects.

Real word Example of JDBC :-

Suppose two places, A and B. People in A don't know the language of people in B and vice versa. To communicate, they need a translator.



- Place A -> Java Application
- Place B -> Database
- Translator -> JDBC Driver (converts Java calls to DB calls and back)
- Road -> Connection (pathway between Java and DB)
- Vehicle -> Statement (carries SQL queries)
- Requirement -> SQL Query
- Small Block (Response) -> ResultSet (data returned from DB)

This entire communication system is what we call Java Database Connectivity (JDBC).

Example :-

```
package JDBCPROGRAMS;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class jdbcday1 {

    public static void main(String[] args) throws ClassNotFoundException, SQLException {
        //1.load class of jdbc driver
        Class.forName("org.postgresql.Driver");
        System.out.println("1.JDBC Driver Loaded!");

        //2.create connection portname,username & password
        //rdbmsdriver://localhost:portnumber/databasename
        String URL="jdbc:postgresql://localhost:5433/java8sept";
        String username="postgres";
        String password="4518";
        Connection con=DriverManager.getConnection(URL,username,password);
        System.out.println("2.Connection is Created");

        //3.create a statement
        String create_table="create table student(id int,name varchar(100))";
        Statement st=con.createStatement();
        System.out.println("3.Statement is Created!");

        //4.execute the statement
        st.executeUpdate(create_table);
        System.out.println("4.Query Executed");

        //5.close the connection
        con.close();
        System.out.println("5.Connection Close!");
    }
}
```

Output :-

```
1.JDBC Driver Loaded!
2.Connection is Created
3.Statement is Created!
4.Query Executed
5.Connection Close!
```